

COMS30026 Design Verification

Verification Tools

Kerstin Eder 

Trustworthy Systems Laboratory

(Acknowledgement: Avi Ziv from the IBM Research Labs in Haifa has kindly permitted the re-use of some of his slides.)



Big Numbers



- 131,000,000 circumference of the earth in feet
- 100,000,000,000 the number of stars in the milky way
- 6,000,000,000,000 the number of miles in one light year
- 20,000,000,000,000 the total net worth in the world, in dollars (M2)
- 1,000,000,000,000,000 number of verification cycles Arm applies to one core

Achieving Automation

Task of Verification Engineer:

- Ensure product does not contain bugs - as fast and as cost-effective as possible.

(... and of Verification Team Leader):

- Select/Provide appropriate tools.
- Select a verification team.
- Decide when cost of finding next bug violates **law of diminishing returns**.
- Parallelism, Abstraction and **Automation** can reduce the duration of verification and, thus, increase productivity.
- Automation reduces the human factor, improves efficiency and reliability.

Verification TOOLS are used to achieve automation.

- Tool providers: Electronic Design Automation (EDA) industry



Annex A (informative)

The Role of Testing in Verification and Validation

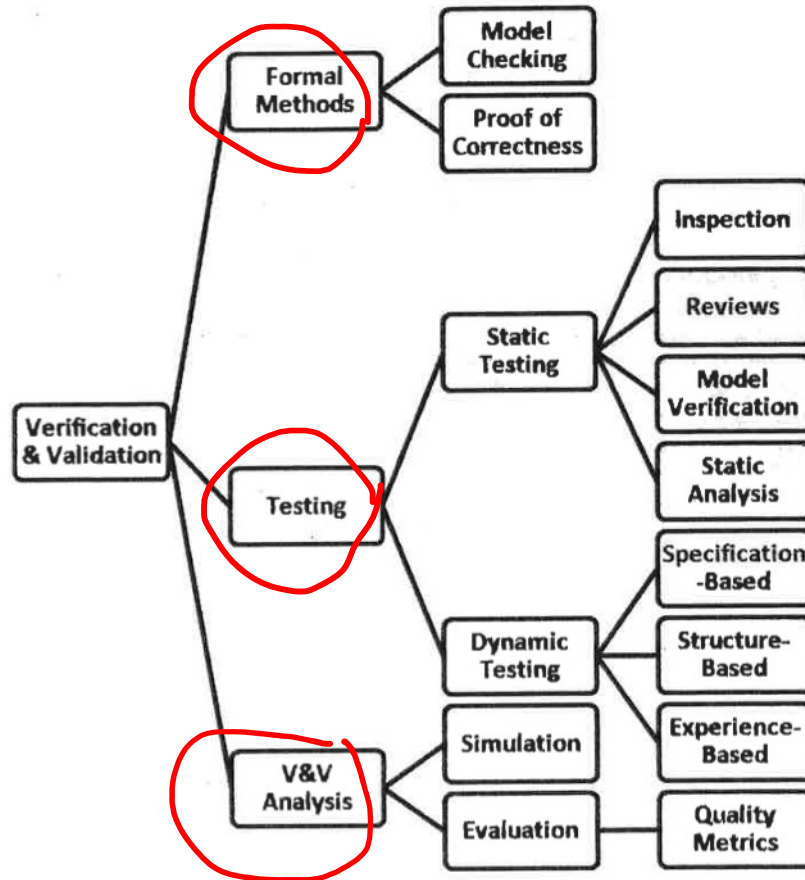


Figure 11 — Hierarchy of Verification and Validation activities

Figure 11 defines the complete nature of verification and validation (V&V) activities. V&V can be done on system, hardware, and software products. These activities and planning are defined and refined in IEEE 1012 and ISO/IEC 12207. Much of V&V is accomplished by testing. The ISO/IEC 29119 standard addresses the Dynamic and Static software testing (directly or via reference), thus covering parts of this verification and validation model. ISO/IEC 29119 is not intended to address all the elements of the V&V model, but it is important for a tester to understand where they fit within this model.



DRAFT INTERNATIONAL STANDARD ISO/IEC DIS 29119-1

ISO/IEC JTC 1

Secretariat: ANSI

Voting begins on
2012-10-09

Voting terminates on
2013-01-09

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION • МЕЖДУНАРОДНАЯ ОРГАНИЗАЦИЯ ПО СТАНДАРТИЗАЦИИ • ORGANISATION INTERNATIONALE DE NORMALISATION
INTERNATIONAL ELECTROTECHNICAL COMMISSION • МЕЖДУНАРОДНАЯ ЭЛЕКТРОТЕХНИЧЕСКАЯ КОМИССИЯ • COMMISSION ELECTROTECHNIQUE INTERNATIONALE

Software and systems engineering — Software testing — Part 1: Concepts and definitions

Ingénierie du logiciel et des systèmes — Essais du logiciel —
Partie 1: Concepts et définitions

ICS 35.080

To expedite distribution, this document is circulated as received from the committee secretariat. ISO Central Secretariat work of editing and text composition will be undertaken at publication stage.
Pour accélérer la distribution, le présent document est distribué tel qu'il est parvenu du secrétariat du comité. Le travail de rédaction et de composition de texte sera effectué au Secrétariat central de l'ISO au stade de publication.

THIS DOCUMENT IS A DRAFT CIRCULATED FOR COMMENT AND APPROVAL. IT IS THEREFORE SUBJECT TO CHANGE AND MAY NOT BE REFERRED TO AS AN INTERNATIONAL STANDARD UNTIL PUBLISHED AS SUCH.
IN ADDITION TO THEIR EVALUATION AS BEING ACCEPTABLE FOR INDUSTRIAL, TECHNOLOGICAL, COMMERCIAL AND USER PURPOSES, DRAFT INTERNATIONAL STANDARDS MAY ON OCCASION HAVE TO BE CONSIDERED IN THE LIGHT OF THEIR POTENTIAL TO BECOME STANDARDS TO WHICH REFERENCE MAY BE MADE IN NATIONAL REGULATIONS.
RECIPIENTS OF THIS DRAFT ARE INVITED TO SUBMIT, WITH THEIR COMMENTS, NOTIFICATION OF ANY RELEVANT PATENT RIGHTS OF WHICH THEY ARE AWARE AND TO PROVIDE SUPPORTING DOCUMENTATION.

© International Organization for Standardization, 2012
© International Electrotechnical Commission, 2012



Tools used for Verification

■ Dynamic Verification:

- Hardware Verification Languages (HVL)
- Testbench automation
- – Test generators
- Coverage collection and analysis
- General purpose HDL Simulators
 - Event-driven simulation
 - Cycle-based simulation (improved performance)
 - Waveform viewers (for debug)
- Hardware accelerators/emulators, FPGAs



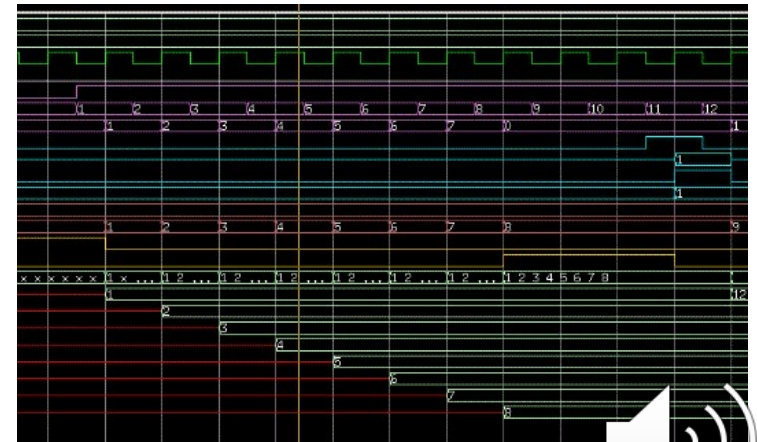
■ Static Analysis / Verification Methods (Formal Methods):

- – Linting Tools
- Equivalence checkers
- – Model checkers
 - Property Specification Languages (ABV)
- Theorem provers

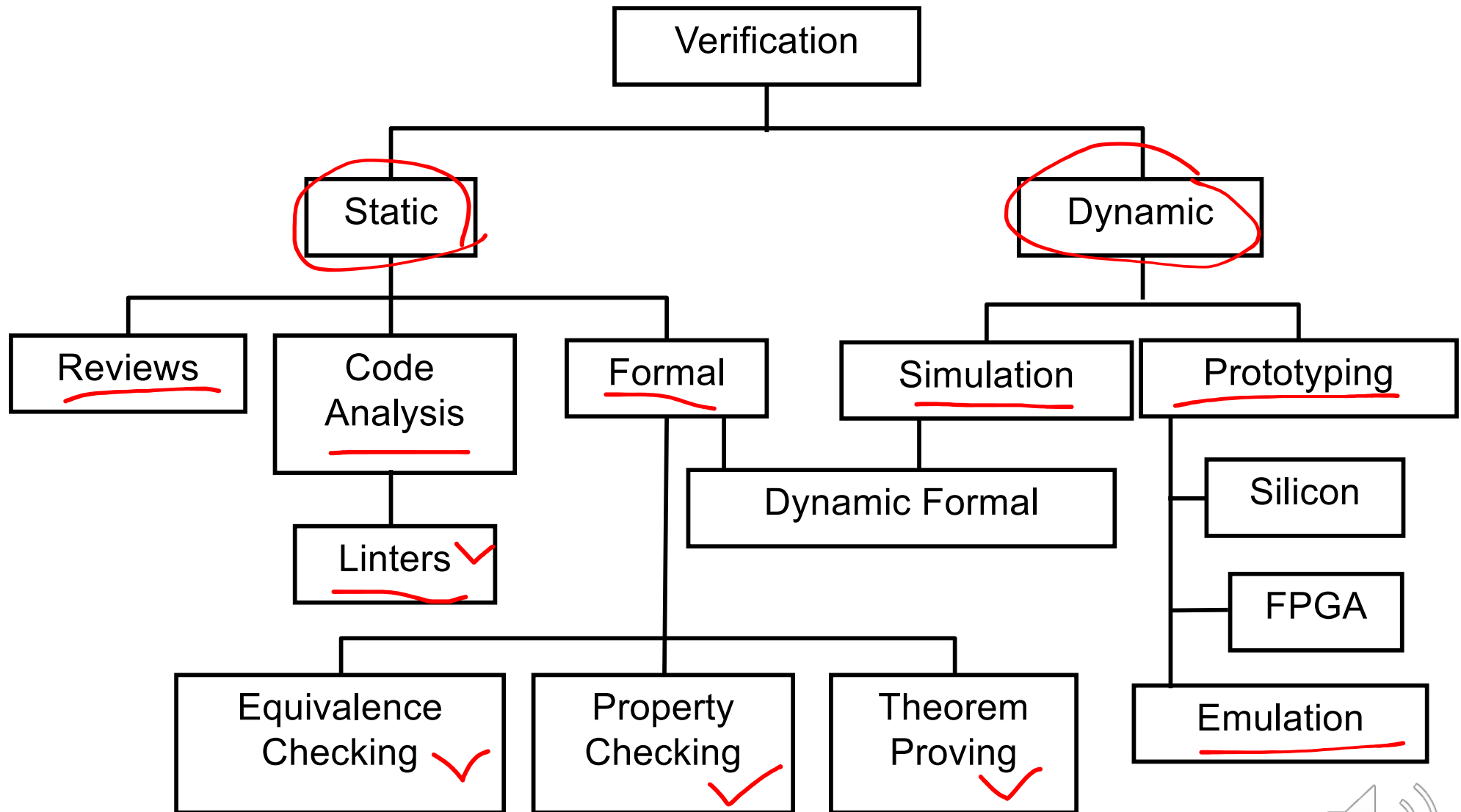
■ Administration:

- Version Control and Issue Tracking
- Metrics
- Data Management and Data Mining related to Metrics

■ Third Party Models



Functional Verification Approaches



Linting Tools

- Linters are **static** checkers.
- Assist in finding common coding mistakes in syntactically correct code
 - Linters exist for software and also for hardware design.
`gcc -Wall / clang-tidy` (When do you use these?)
- Linters only identify certain classes of problems
 - Many false positives (i.e. false alarms) are reported.
 - Use a **filter** program to reduce these.
 - Careful - don't filter true positives though!
- Linters can assist in enforcing **coding guidelines!**
 - Rules for coding guidelines can be added to linter.

Is there any merit in using a linter alongside simulation-based testing?



Tools for Simulation- Based Verification

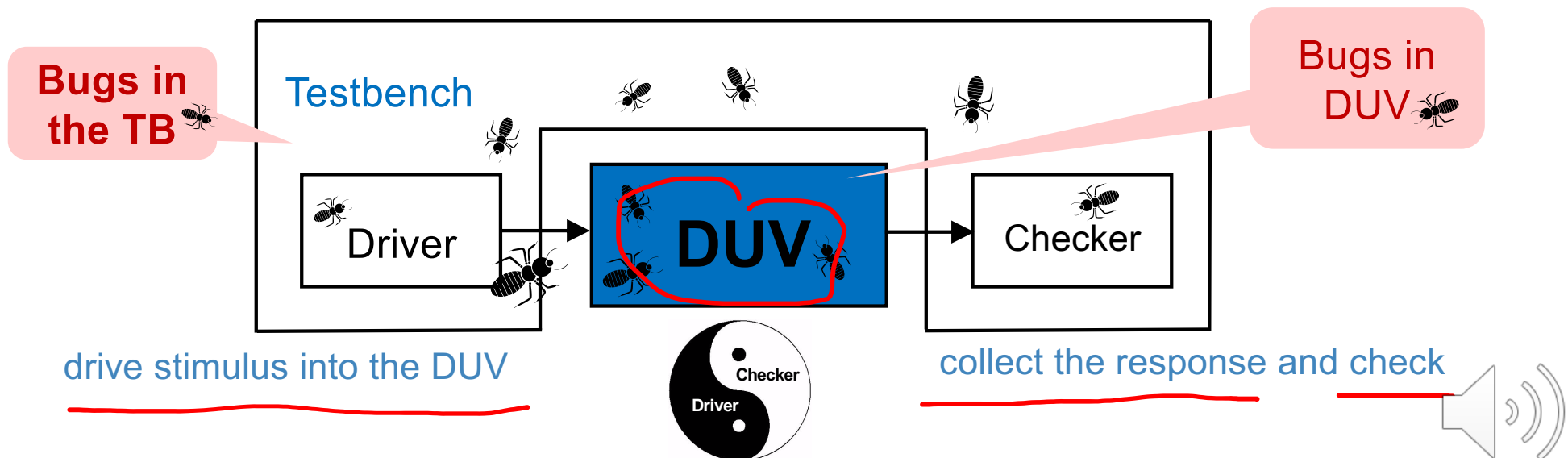
Simulators



What is a Testbench?

*“Code used to create a predetermined **input sequence** to a design, and to then observe the **response**.”*

- Generic term used differently across the industry.
- Always refers to a test case/scenario.
- Traditionally, a testbench refers to code **written in a Hardware Description Language (VHDL, Verilog)** at the top level of the design hierarchy.
- **A testbench is a “completely closed” system:**
 - No inputs or outputs.
 - Effectively a model of the universe as far as the design is concerned.



Simulation-based Design Verification

- Simulate the design (not the implementation) **before** fabrication.
- **Simulating the design relies on simplifications:**
 - Functional correctness/accuracy can be a problem.

Verification Challenge: "What input patterns to supply to the Design Under Verification (DUV) ..."

- Simulation requires **stimulus**. It is dynamic, not just static!
- Requires to reproduce the environment in which the DUV will be used.
 - **Testbench** (Remember: Verification vs Testing!)



Simulation-based Design Verification

- Simulate the design (not the implementation) **before** fabrication.
- **Simulating the design relies on simplifications:**
 - Functional correctness/accuracy can be a problem.

Verification Challenge: *"What input patterns to supply to the Design Under Verification (DUV) ..."*

- Simulation requires **stimulus**. It is dynamic, not just static!
- Requires to reproduce the environment in which the DUV will be used.
 - **Testbench** (Remember: Verification vs Testing!)

Verification Challenge (continued): *"... and knowing what is expected at the output for a properly working design."*

- **Simulation outputs are checked externally** against the design intent (i.e. against the specification)
 - Errors cannot be proven not to exist!

"Testing shows the presence, not the absence of bugs."

[Edsger W. Dijkstra]

(e.g. *"The Humble Programmer"*)

see also the famous EWD manuscript
<https://www.cs.utexas.edu/users/EWD/ewd03xx/EWD340...>



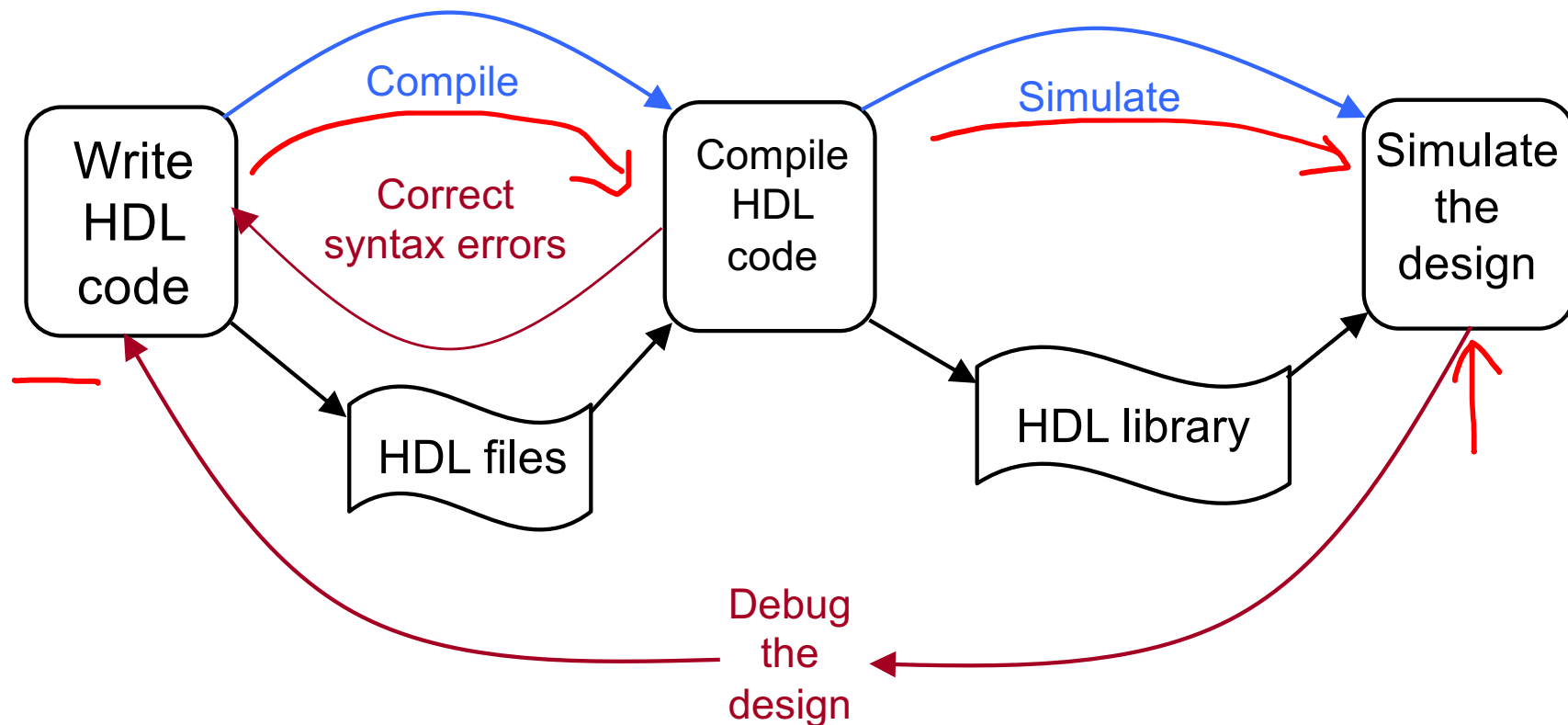
General HDL Simulators

- Most Popular Simulators in Industry
 - **Siemens ModelSim/Questa** ←
 - **Cadence NCSim** ←
 - Synopsys VCS
- Provide full language support
 - "EVENT DRIVEN" algorithms
- VHDL's execution model is defined in detail in the IEEE LRM (Language Reference Manual)
- Verilog's execution model was for a long time defined by Cadence's Verilog-XL simulator ("reference implementation"); it is now an IEEE standard, e.g. Verilog 2005



Simulation based on Compiled Code

- To simulate with **ModelSim/Quarta**:
 - Compile HDL source code into a library.
 - Compiled design can be simulated.



Two types of simulators:

event-based and
cycle-based



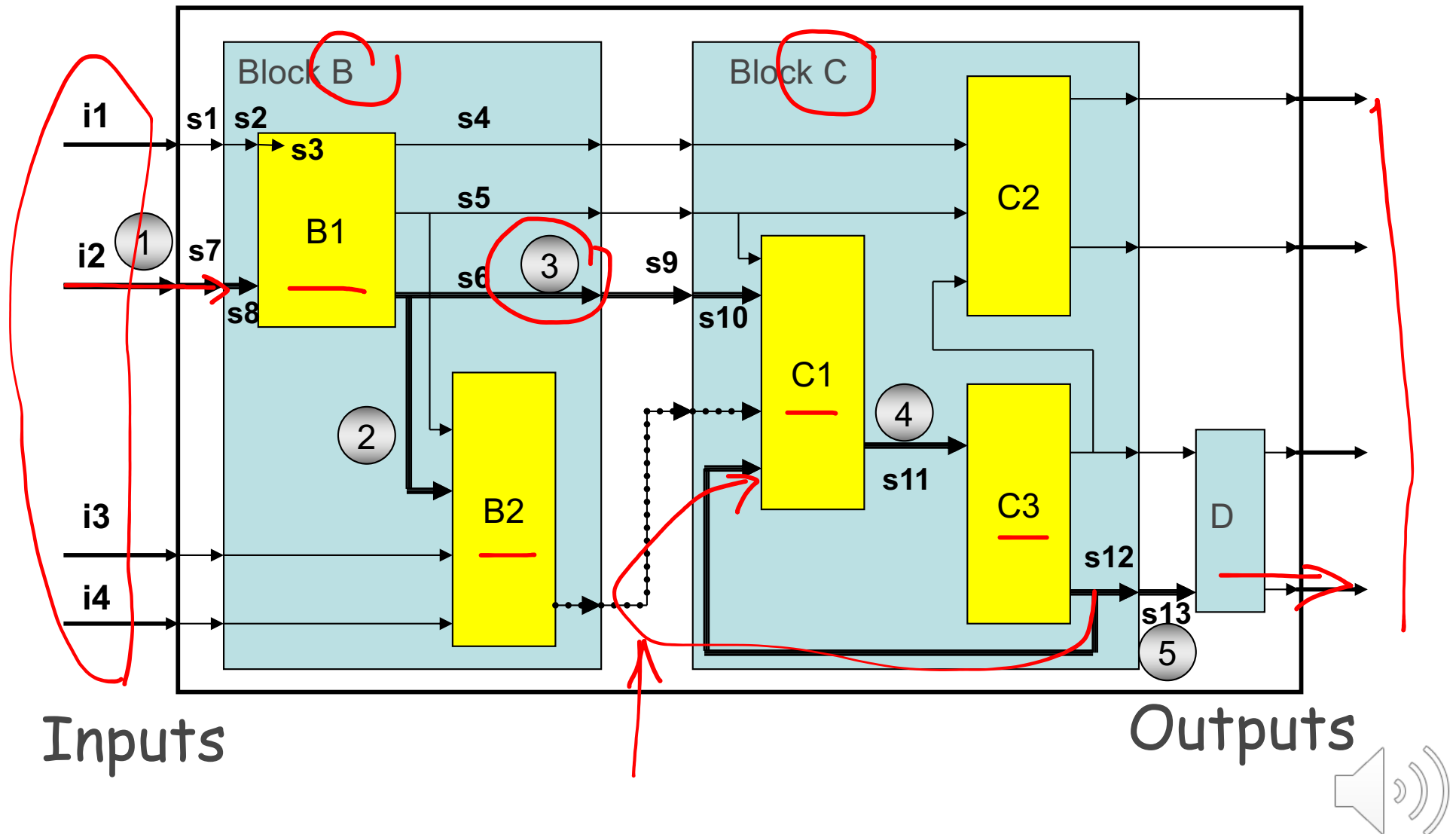
Event-based Simulators

Event-based simulators are driven based on **events**. 😊

- Outputs are a function of inputs:
 - The outputs change only when the inputs do.
 - **The event is the input changing.**
 - An event causes the simulator to re-evaluate and calculate new output.
- Outputs (of one block) may be used as inputs (of another) ...



Event Flow Example



Event-based Simulators

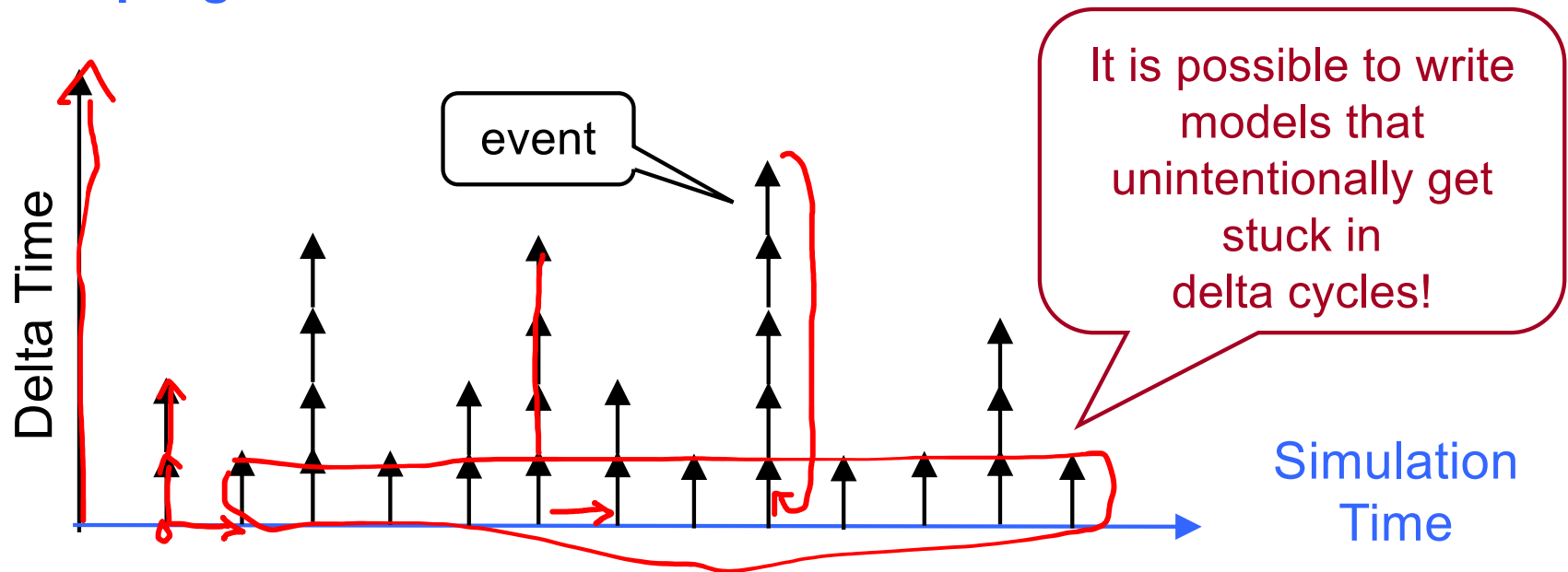
Event-based simulators are driven based on **events**. 😊

- Outputs are a function of inputs:
 - The outputs change only when the inputs do.
 - **The event is the input changing.**
 - An event causes the simulator to re-evaluate and calculate new output.
- Outputs (of one block) may be used as inputs (of another) ...
- **Re-evaluation happens until no more changes propagate through the design.**
- Zero delay cycles are called **delta cycles!**



Delta Cycles

- **Event propagation** may cause the assignment of new values after **zero** (simulation time) delays.
 - (Remember, this is only a **model** of the physical circuit.)
- Although **simulation time** does **not advance**, the **simulation makes progress**.



- NOTE: Simulation progress is first along the zero/delta-time axis and then along the **simulation time axis**.

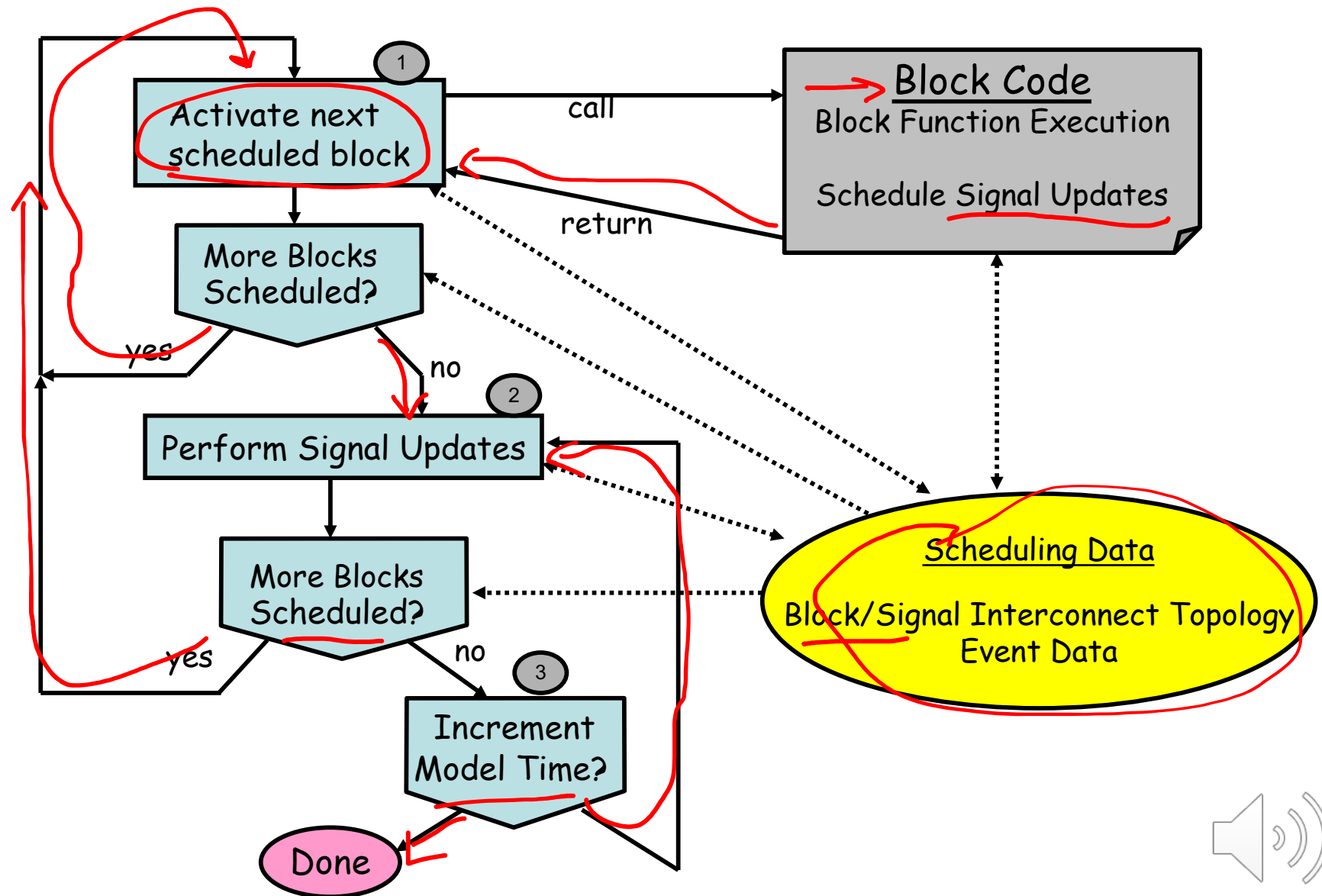


Event-Driven Simulation Principles

- The event simulator maintains many lists:
 - A list of all **atomic** executable blocks
 - Fanout lists: A data structure that represents the interconnect of the blocks via signals
 - **A time queue**
 - records the points in time when events happen
 - **Event queue**
 - one queue pair for each entry in the time queue
 - Signal update queue
 - Computation queue
- The simulator needs to process all these queues at simulation time.



Core Algorithm of an Event-Driven Simulation Engine



Simulation Speed

What is holding us back?

Speedup strategies



Improving Simulation Speed

- The most obvious **bottle-neck** for functional verification is **simulation throughput**
- There are several ways to improve throughput
 - Parallelization
 - Compiler optimization techniques
 - Changing the level of abstraction

SSE: further information on the above three methods is provided in the slides and explained in more detail in the pre-recording on BB unit page

- Design methodology-based subsets of HDL
 - **Cycle-based simulation**
- Special simulation machines



Two types of simulators:

event-based and
cycle-based



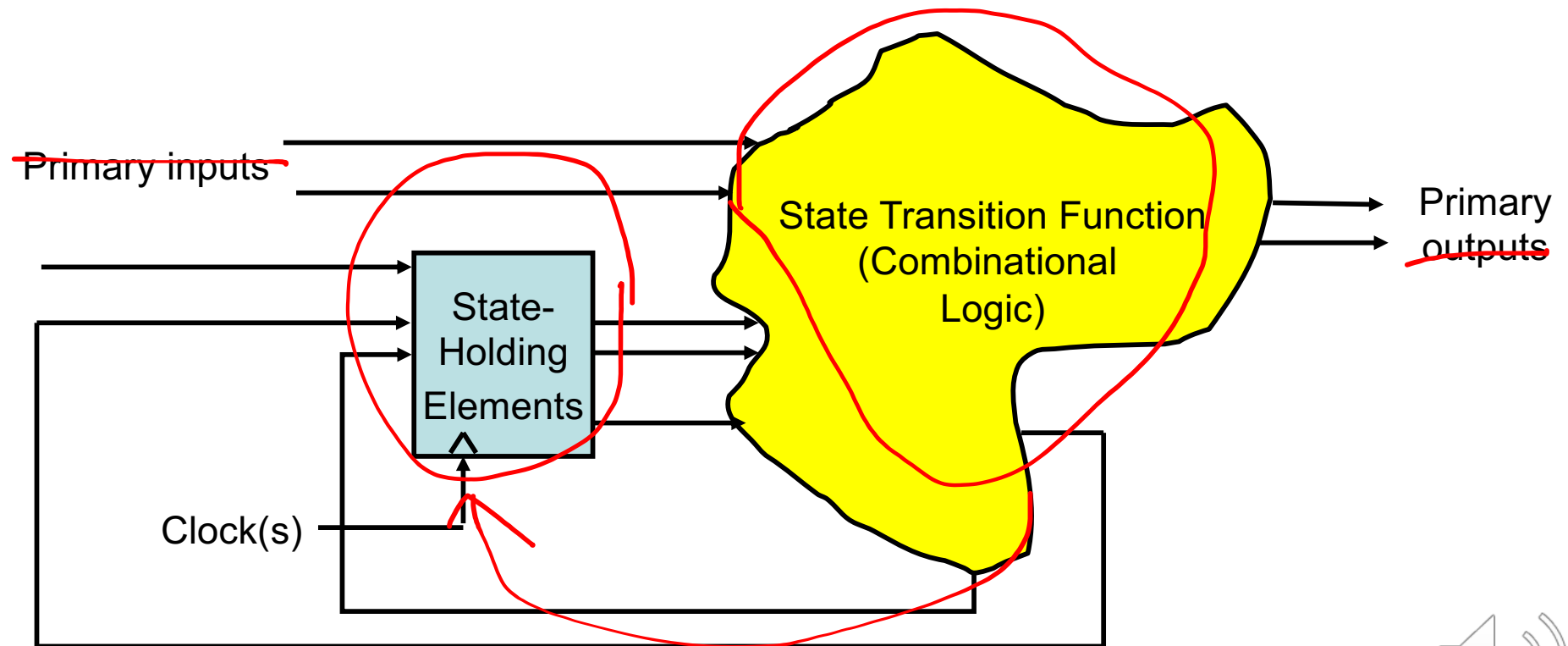
Two types of simulators:

event-based and
cycle-based



Synchronous Design Methodology

- The design is comprised of
 - State-holding (storage) elements
 - Combinational logic for state transition function

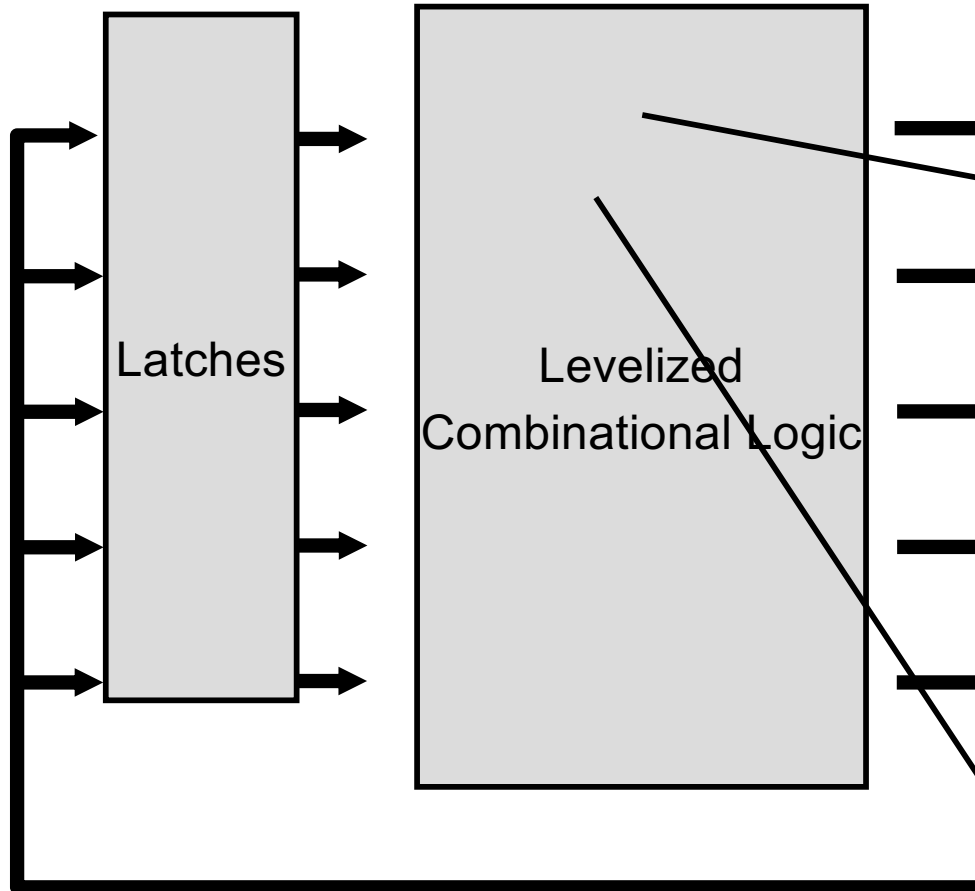


Cycle-Based Model Build Flow

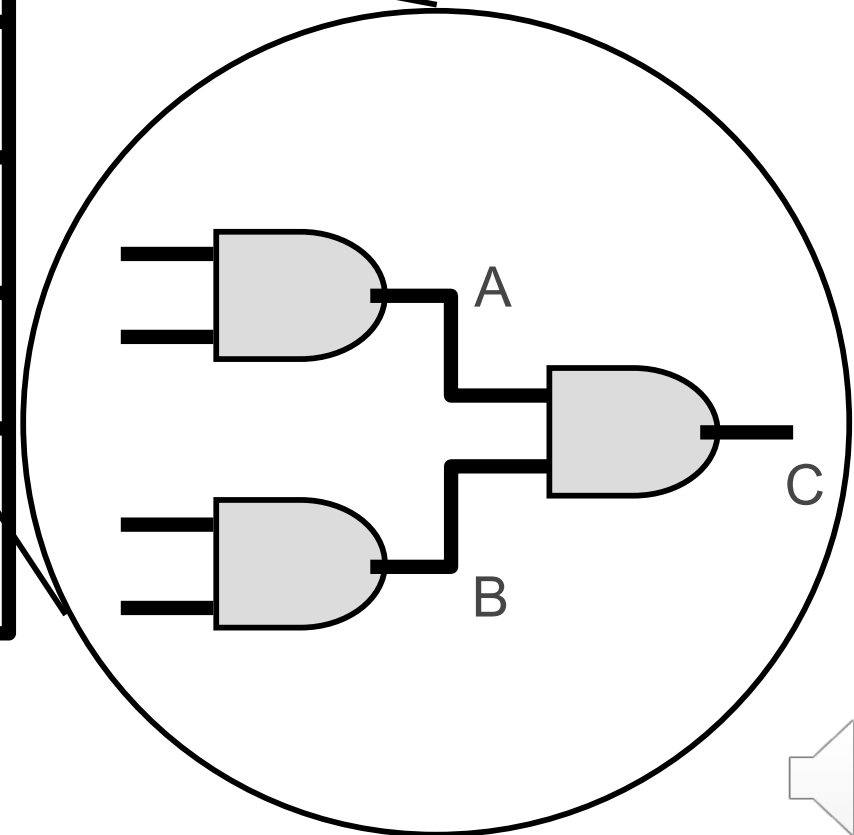
- Language compile – synthesis-like process
 - Simpler because of missing physical constraints
 - Logic mapped to a non-cyclic network of Boolean functions
 - Hierarchy is preserved during language compilation
- Flatten hierarchy – crush design hierarchy to increase optimization potential
- Optimization – minimize the size of the model to increase simulation performance
- Levelize logic
- Translate to dedicated instructions



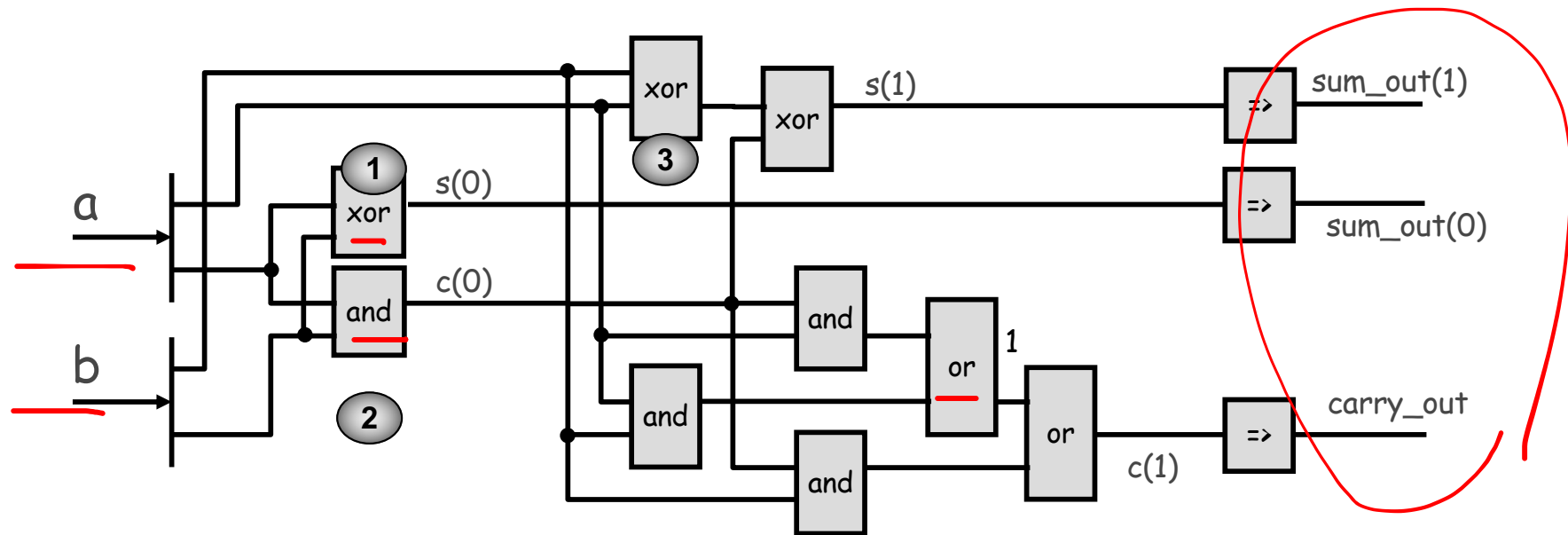
Model Build – Levelization



Logic is ordered into levels so that the order of evaluation is correct.
e.g., A and B are computed before C.



Translate to Instructions



- 1 Load temp1, a(0)
Load temp2, b(0)
Xor temp1, temp2, temp3
Store temp3, s(0)
- 2 And temp1, temp2, temp3
Store temp3, c(0)
Load temp1, a(1)
Load temp2, b(1)
- 3 Xor temp1, temp2, temp3
...

We can convert every Boolean function into a minimal set (~4 or better) of dedicated instructions



Parallelism in Generated Code

- Word-level operations can be easily parallelized

$A(0 \text{ to } 31) \leq B(0 \text{ to } 31) \text{ and } C(0 \text{ to } 31) \text{ and } D(0 \text{ to } 31)$

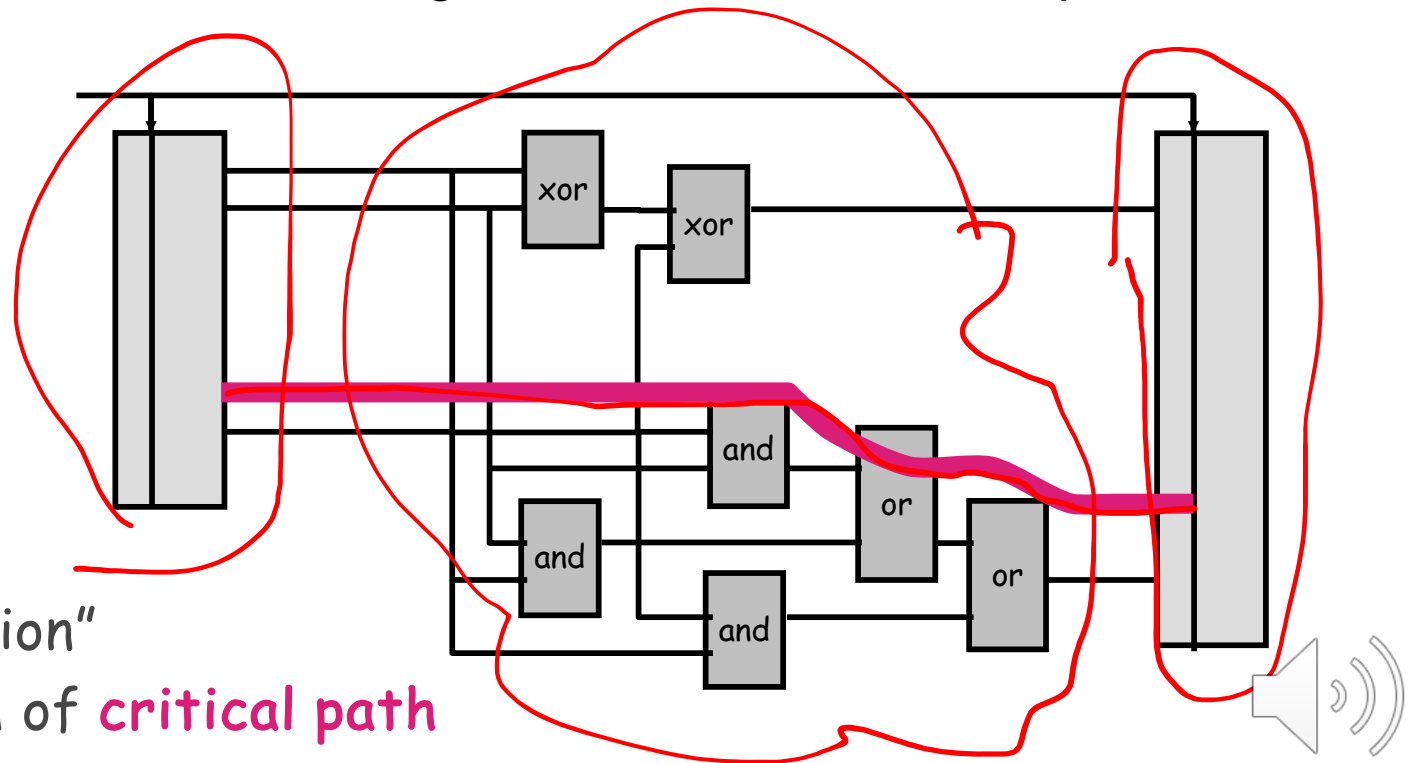
Is translated into

```
LoadWord R1, B(0 to 31)
LoadWord R2, C(0 to 31)
AND R1, R1, R2
LoadWord R2, D(0 to 31)
AND R1, R1, R2
StoreWord R1, A(0 to 31)
```



Speed of Cycle-Based Simulation

- Clock the design only as fast as the longest possible combinational delay path settles before the cycle is over
- Cycle time depends on the longest topological path
 - Longest topological path can be calculated using static analysis w/o using simulation -> stronger result w/o simulation patterns



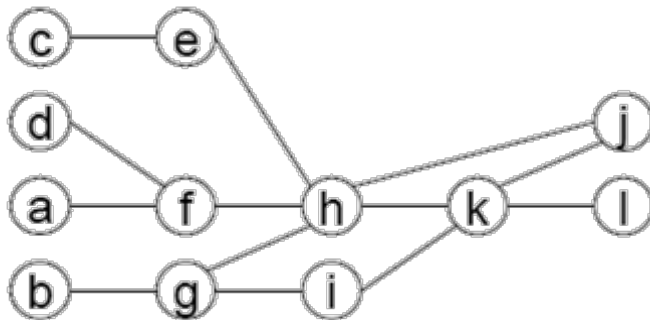
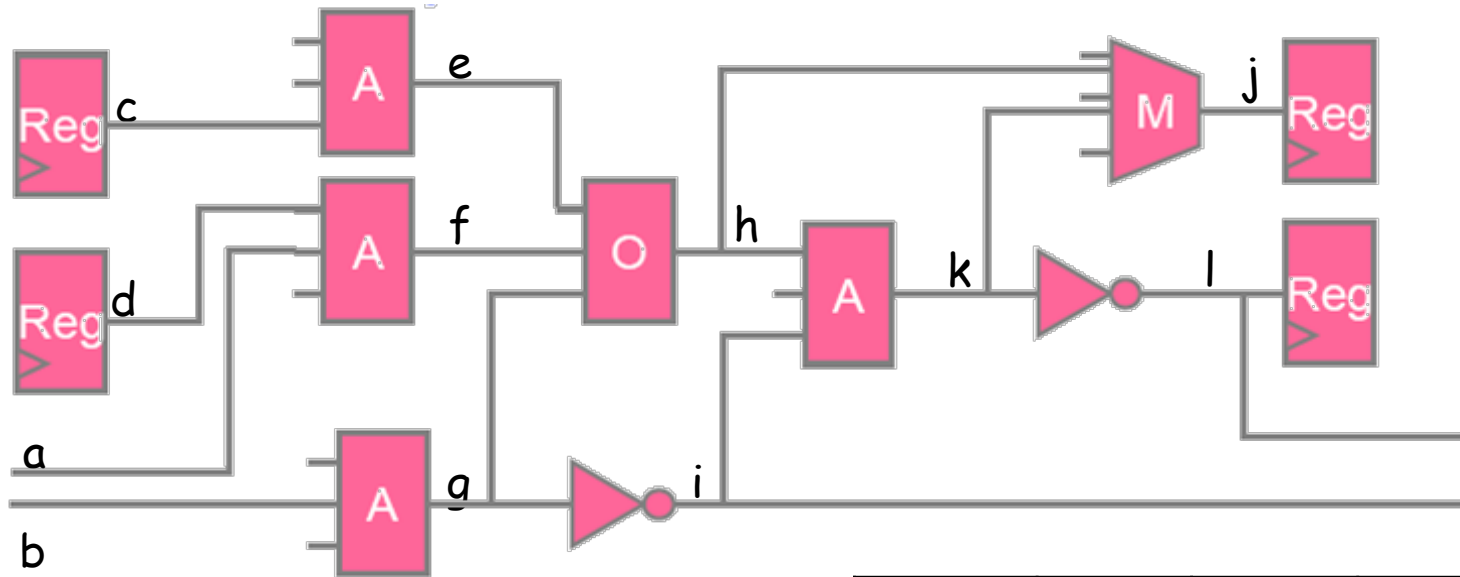
clock cycle "duration"
depends on length of **critical path**

Hardware Acceleration

- Programs created for cycle-based simulation are very simple
 - Small set of instructions
 - Simple control – no branches, loops, functions
- Operations at the same level can be executed in parallel (Levelization)
- Hardware acceleration exploit these facts for fast simulation by utilizing
 - Very large number of small and simple special-purpose processors
 - Efficient communication and scheduling



Scheduling Example

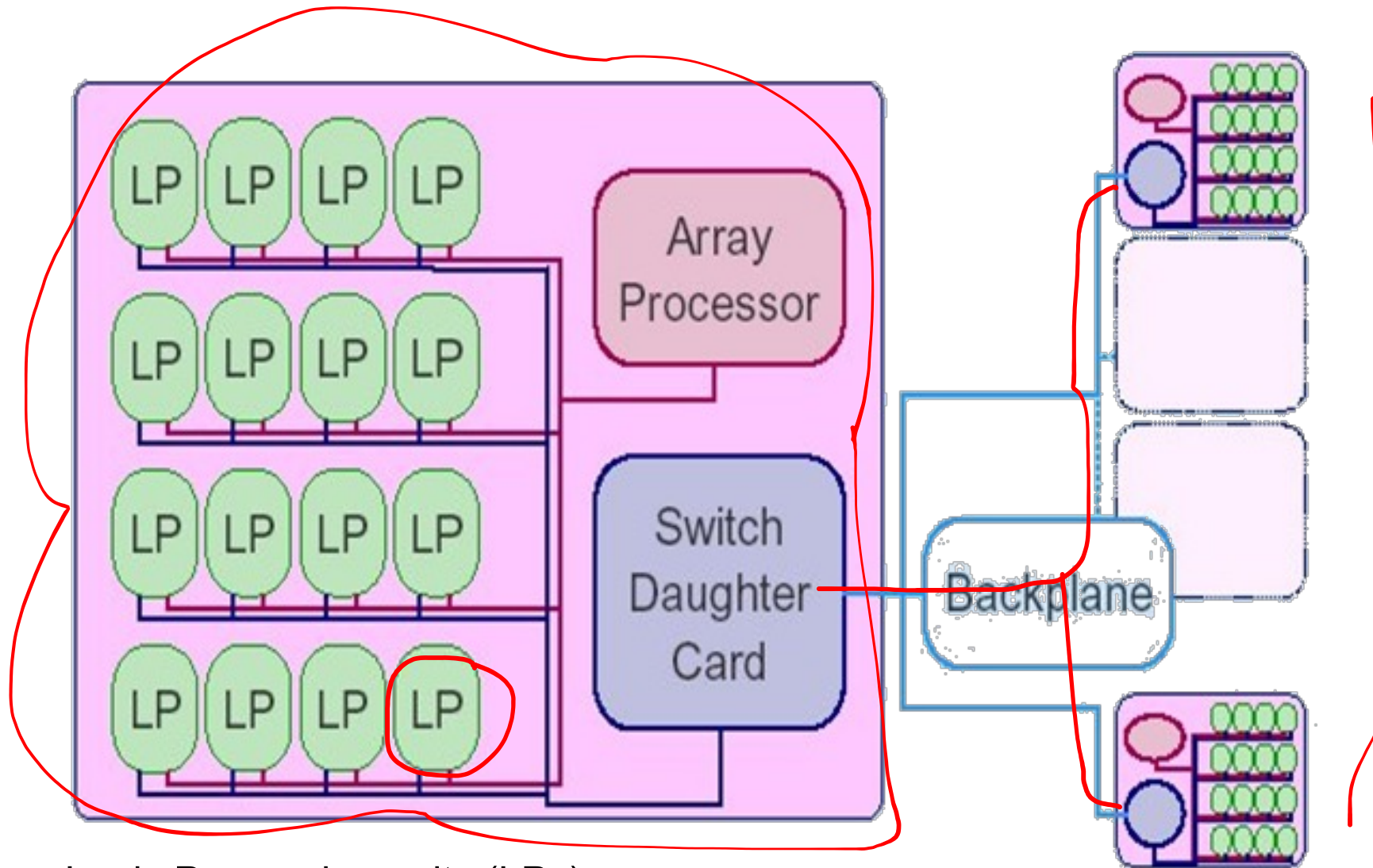


	EP1	EP2	EP3	EP4
Step1	b	a	d	c
Step2	g	f		e
Step3	i	h		
Step4	k			
Step5	j	l		

12 steps serial, 5 steps parallel



Accelerator Basic Structure



Logic Processing units (LPs):

These are special purpose processing units, that are much faster than CPUs.



Principle of Operation

- Compiler transforms combinational logic into Boolean operations
- Compiler schedules inter-processor communications using a fast broadcast technique
- Performance dictated by
 - Number of processors (Logic Processing units)
 - Number of levels in the design



Simulation Speed Comparison

→	Event-based Simulator	1
→	Cycle-based Simulator	20
	Event-driven cycle Simulator	50
→	Acceleration	<u>1000</u>
	Emulation	100000



Simulation Speed Comparison

<u>Event-based Simulator</u>	1
Cycle-based Simulator	20
Event-driven cycle Simulator	50
Acceleration	1000
Emulation	100000





Verification Futures Conference UK 2025

Tue 1 Jul 2025 8:30 AM - 5:00 PM BST

📍 Online, Microsoft Teams



*Datacenter Performance. Edge Efficiency.
Accelerating Inference, Everywhere.*

Multisim: simulate RTL with real multi-threaded speed

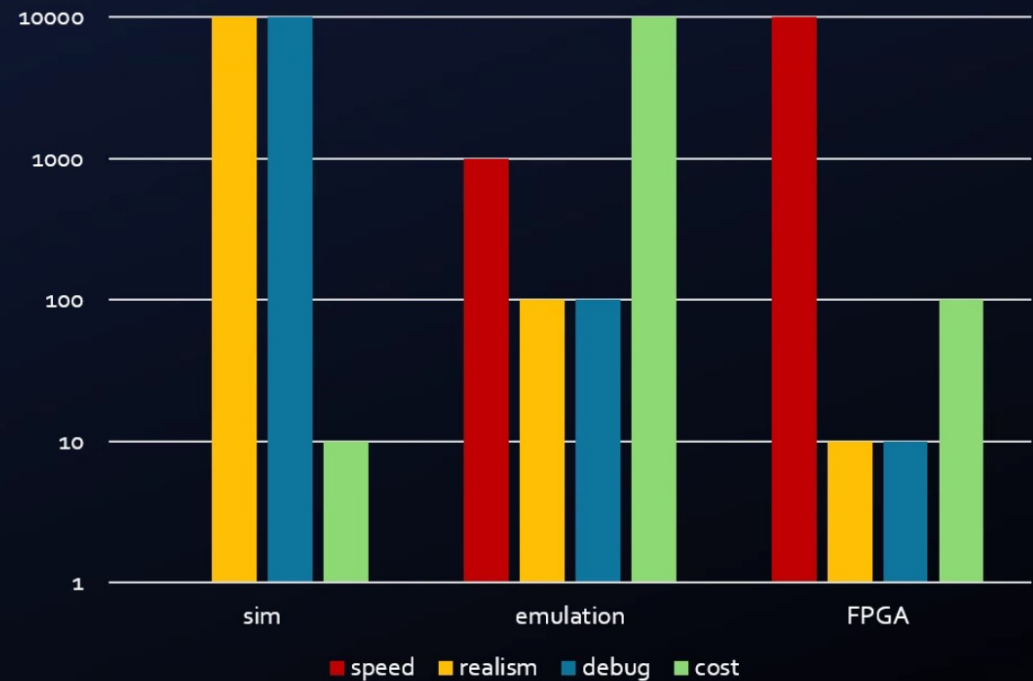
Verification Futures Conference 2025 UK - 1st July 2025

Antoine Madec

Verification platform landscape

Current verification platform landscape

- Big projects are too slow for classic simulation
 - Emulation, models, etc, are a must
 - The right platform for the right tests for the right interfaces
 - Trade-offs are inevitable
 - Speed
 - Realism
 - Debugability
 - Platform bring-up time
 - Price



- It is our job to choose the right platform

<https://www.tessolve.com/verification-futures/vf2025-uk/multisim-simulate-your-rtl-with-real-multi-threaded-speed/>



Putting processor verification into context....



1,000,000,000,000,000

The number of verification cycles Arm uses when verifying an Arm core

- SystemVerilog simulator executes 2,000 cycles / second
=> 15,000 SystemVerilog simulators running for 1 year
- HW emulator or FPGA runs at 1,000,000 cycles / second
⇒ 30 years of running needed...
- OK – so this is for a high performance OoO, MP, VM core (full apps processor)
 - Embedded processors will be an order of magnitude less...

Verification Languages

Raising the level of
abstraction



Verification Languages

- Need to be designed to address **verification principles**.
- Deficiencies in RTL languages (HDLs such as Verilog and VHDL):
 - **Verilog** was designed with focus on describing low-level hardware structures.
 - No support for **data structures** (records, linked lists, etc).
 - Not object/aspect-oriented.
 - Useful when several team members develop testbenches.
 - VHDL was designed for large design teams.
- Limitations of HDLs inhibit **efficient** implementation of verification strategy.
- High-level verification languages are (currently):
 - **System Verilog** ←
 - IEEE 1800 [2005] Standard for System Verilog- Unified Hardware Design, Specification, and Verification Language
 - e-language used for Cadence's Specman Elite [IEEE P1647]
 - (Synopsys' Vera, System C)



Features of High-Level Verification Languages

- Raising the level of abstraction:
 - From bits/vectors to high-level data types/structures
 - lists, structs, scoreboards including ready made functions to access these
- Support for building the verification environment
 - Enable testbench automation
 - Modularity
 - Object/aspect oriented languages
 - Libraries (VIP) to enable re-use
- Support for test generation
 - Constrained random test generation features
 - Control over randomization to achieve the target values
 - Advanced: Connection to DUV to generate stimulus depending on DUV state
- Support for coverage
 - Language constructs to implement functional coverage models

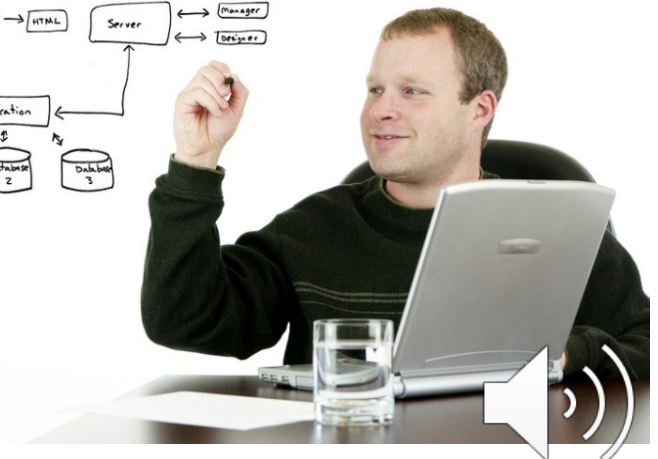
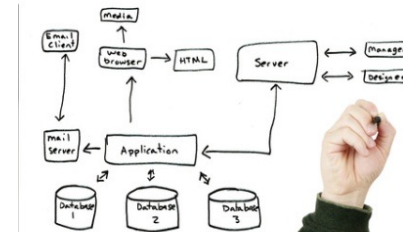


Any other *verification* Languages?

Tommy Kelly, CEO of Verilab:

“Above all else, the Ideal Verification Engineer will know how to construct software.”

- Toolkit contains not only Verilog, VHDL, SystemVerilog and e, but also Python, Lisp, MySQL, Java, ... ☺



Verification Tools (other)

Waveform Viewers

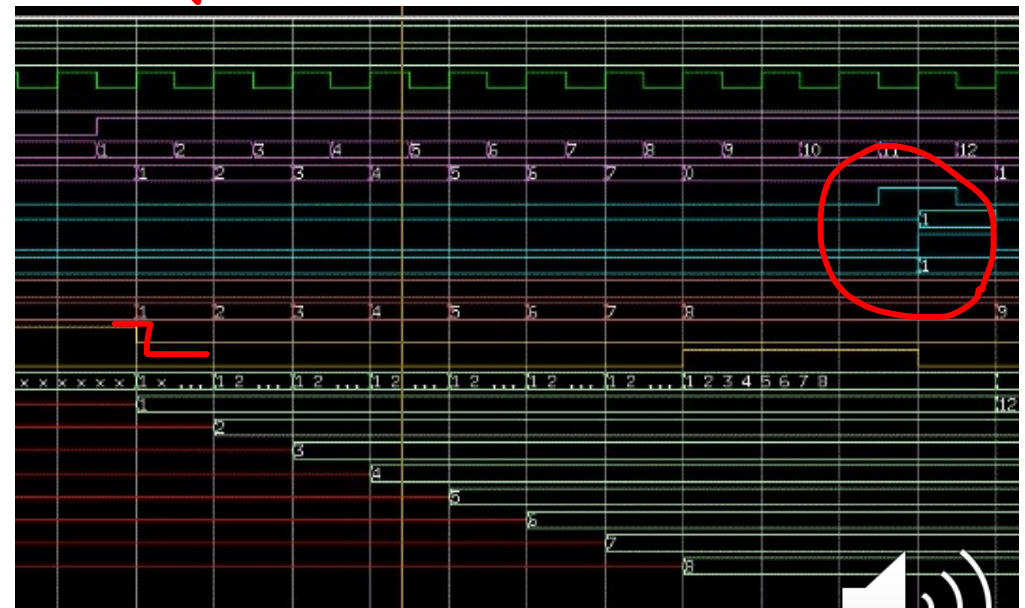
Third Party Models

Metrics



Waveform Viewers

- **Waveform viewers** often come as part of a simulator.
- **Most common verification tools used...**
 - Used to **visually inspect** design/testbench/verification environment.
 - Recording waves decreases performance of simulator. (Why?)
- **Don't use waveform viewers to determine whether the DUV passes or fails a test.**
 - **Why not?**
- **Can use waveform viewer for debugging.**
 - Sophisticated debuggers link from the wave directly to the source code and highlight the related logic.



Third Party Models

- Chip needs to be verified in its **target environment**.
 - Board/SoC Verification
- Do you develop or purchase behavioural models (specs) for board parts?
 - Buying them may seem expensive!
 - Ask yourself:
“If it was not worth designing on your own to begin with, why is writing your own model now justified?”
 - The model you develop is not as reliable as the one you buy.
 - The one you buy is used by many others - not just yourself.
- Remember: In practice, it is often more expensive to develop your own model to the **same degree of confidence** than licensing one.



Metrics

- Not really verification tools - but managers love metrics and measurements!
 - Managers often have little time to personally assess progress.
 - They want something measurable.
- **Coverage** is one metric - will be introduced later.
- **Other metrics include, e.g.:**
 - Number of lines of code
 - Ratio of lines of code
(between design and verification)
 - Drop of source code changes
 - Number of outstanding issues



Summary

- Basic introduction to testbenches

We have studied

- Verification Tools
 - including an outlook on Verification Languages
- Linting tools – static checkers
- Simulators: event-based and cycle-based
 - How to improve the performance of simulation-based verification
 - Compared the speed of different simulation techniques
 - Dangers of using waveform viewers as checkers



Next

- Recordings of lectures (about 2h - 3h per week)

Week 1:

- ✓ Introduction to Design Verification
- ✓ Verification Hierarchy
- ✓ Driving & Checking

- uobdv.github.io/Design-Verification/
shows a **weekly schedule of topics** to watch
BEFORE the next session, ideally
- Recordings are available from Blackboard unit page

- Tasks for you this week:
 - **Attend the lab session** on Thursday to set up access to the EDA tools
 - **Paper review “*The limits of correctness*”**

Paper review

*Brian Cantwell Smith. 1985. The limits of correctness.
SIGCAS Comput. Soc. 14,15, 1,2,3,4 (Jan 1 1985), 18–26.
DOI: <https://doi.org/10.1145/379486.379512>*

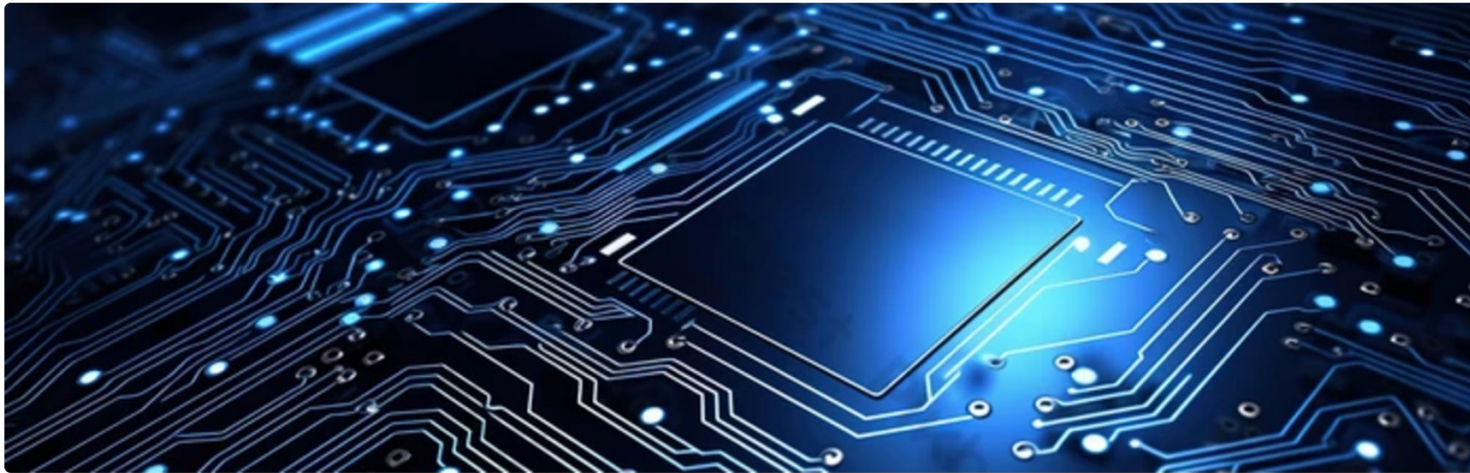
THE LIMITS OF CORRECTNESS[†]

Brian Cantwell Smith*

- *Identify the main lines of argument*
- *Why does the author question the notion of “correctness”?*
- *What are the two or three key take-away messages for you?*

Over the last ten years, the Defense Department has spent many millions of dollars on a new computer technology called "program verification" - a branch of computer science whose business, in its own terms, is to "prove programs correct". Pro-

And my answer, to give away the punch-line, is no. For fundamental reasons - reasons that anyone can understand - there are inherent limitations to what can be proven about computers and computer programs. Although program verification is an important new technology, useful, like so many



WEBINAR: Verification Productivity Trends

Select tickets

Fri 24 Oct 2025 12:00 PM - 1:00 PM CDT

📍 Online

Verification Productivity Trends

Verification teams are constantly pressured to deliver faster, with fewer bugs, and on increasingly complex designs. This webinar will explore how “shifting left” is transforming verification practices, making them more productive, predictable, and scalable.

We'll discuss the latest productivity trends in verification, and hear perspectives from leading companies on how methodologies, tools, and best practices are evolving.

WEBINAR: Verification Productivity Trends

Fri 24 Oct 2025 12:00 PM - 1:00 PM CDT

📍 Online

Select tickets

<https://www.tickettailor.com/events/alpinumconsulting/1836085>

Registration and attendance is completely FREE!

Understanding Random Stability in SystemVerilog and UVM

Wednesday, November 03, 2021

Time: 10-11am (GMT) 11-12pm (CET) 3.30-4.30pm (IST)

[REGISTER NOW »](#)

Time: 10-11am (PDT) 11-12pm (MDT) 12-1pm (CDT) 1-2pm (EDT)

[REGISTER NOW »](#)

Webinar Overview:

A common issue with constrained random simulation is being able to reproduce random stimulus for debug purposes and for locking down regressions test suites. This is especially problematic when the source code needs to be modified and is known in SystemVerilog as random stability.

In this webinar, we explain:

- Random stability in SystemVerilog and in UVM, the Universal Verification Methodology
- The pitfalls of poor random number generation and seeding
- How to ensure that simulation results are reproducible in native SystemVerilog and in UVM through a careful presentation of hierarchical seeding, manual seeding, thread stability, and object stability
- Exactly what changes you can and cannot make to UVM code without disturbing the random stimulus generation.

Examples written in the IEEE Std 1800™ SystemVerilog language and UVM 1.2 will be shown running on the [Questa Advanced Simulator](#) from Siemens.

This training webinar is presented by Doulos Senior Member Technical Staff, Matthew Taylor and will consist of a one-hour broadcast with interactive Q&A available to attendees throughout.

The Keys to SystemC & TLM-2.0: How to be Successful

Friday, November 05, 2021

Time: 10-11am (GMT) 11-12pm (CET) 3.30-4.30pm (IST)

[REGISTER NOW »](#)

Time: 10-11am (PDT) 11-12pm (MDT) 12-1pm (CDT) 1-2pm (EDT)

[REGISTER NOW »](#)

Webinar Overview:

SystemC has become well-established as the language of choice for system modeling and virtual platform creation and integration, and is now being applied successfully for high level synthesis. SystemC models also frequently appear as reference models in the hardware verification flow.

This session is aimed at hands-on hardware or software engineers who might know Verilog or C but have no previous experience of SystemC. It will explain what you need to know to be successful with SystemC by exploring some fundamental questions including:

- What is SystemC and how is it used?
- What does modern SystemC code look like?
- What differentiates SystemC from SystemVerilog?
- What use cases best fit SystemC?
- What is TLM?
- Where can I learn more?

The session will be presented by David C Black, Doulos Senior Member of Technical Staff and co-author of "SystemC: From the Ground Up". It will consist of a one-hour session, (see below for details) and will be interactive with Q&A participation from delegates.

